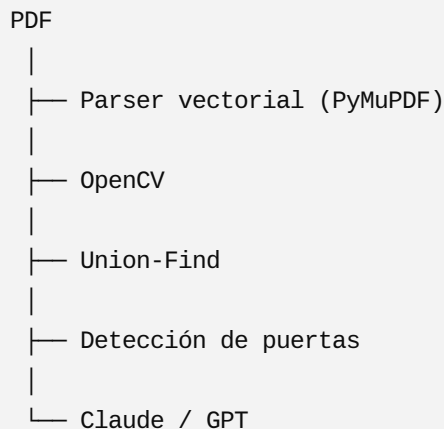


Creo que **FloorplanVLM encaja sorprendentemente bien con ArchiCheck**, pero **no como reemplazo** de lo que ya construyeron. Lo usaría como una **tercera fuente de verdad** dentro de un pipeline híbrido.

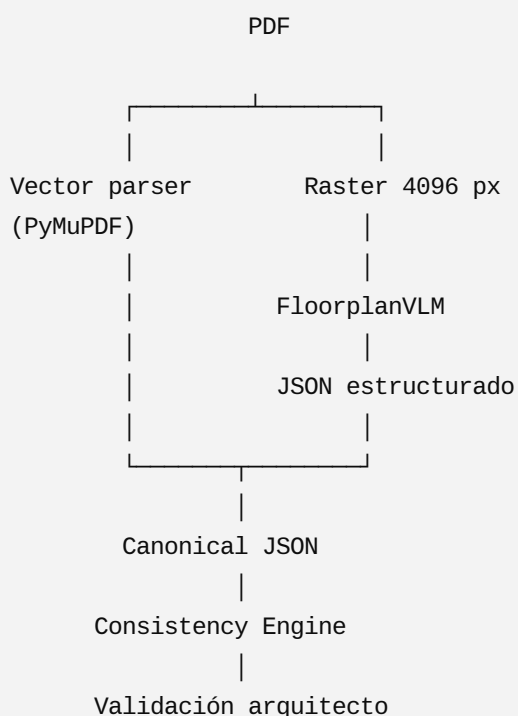
Lo más interesante del paper no es el VLM en sí, sino el cambio de paradigma: en vez de segmentar píxeles y luego reconstruir geometría con heurísticas, el modelo genera directamente una **secuencia JSON que representa la topología completa del plano** ("pixels-to-sequence"). Además, los autores afirman que el modelo generaliza mejor a geometrías no Manhattan (muros oblicuos y arcos), algo relevante para ustedes. [✖ a... +1](#)

Yo NO lo pondría al principio del pipeline

Hoy tienes algo como:



Yo evolucionaría a:



Es decir, **no reemplazaría** tu extracción vectorial.

La usaría para detectar dónde ambas fuentes discrepan.

Lo que más me interesa

El paper dice explícitamente que la salida es un JSON estructurado. ✖ a... +1

Si realmente tiene una forma parecida a:

```
JSON

{
  "walls": [...],
  "doors": [...],
  "rooms": [...],
  "windows": [...]}

```

entonces puedes escribir un adaptador único.

Por ejemplo:

```
Python

floorplanvlm_json

↓

canonical_json.py

↓

{
  "spaces": [],
  "walls": [],
  "doors": [],
  "windows": []
}
```

Ejecutar

Desde ese momento tu motor normativo no sabe si la información provino de:

- PyMuPDF
- CubiCasa
- FloorplanVLM
- un futuro modelo propio

Donde creo que más valor tendría

Piensa en el problema de las siete puertas que mencionaste.

Hoy tienes:

Parser

↓

0 puertas

Con FloorplanVLM podría ocurrir:

Parser

↓

0 puertas

FloorplanVLM

↓

7 puertas

Eso no significa aceptar automáticamente las siete.

Significa generar un conflicto:

JSON

```
{
  "door_conflicts": [
    {
      "location": [351, 712],
      "vector_engine": "missing",
      "floorplanvlm": "door"
    }
  ]
}
```

La interfaz del arquitecto podría mostrar:

"El modelo detectó una puerta que el parser vectorial no encontró."

Eso es muchísimo más útil que confiar ciegamente en cualquiera de los dos.

Incluso lo usaría para autoajustar el parser

Aquí es donde creo que hay una oportunidad interesante.

Hoy ajustan manualmente:

- tolerancias
- ángulos
- radios
- filtros

Yo probaría algo como:

Parser

↓

5 puertas

FloorplanVLM

↓

7 puertas

↓

Comparador

↓

¿Qué cambió?

↓

Buscar arcos

Buscar Bézier

Buscar color

Buscar layer

Es decir, usar FloorplanVLM como generador de hipótesis para mejorar el parser determinista.

Donde NO lo usaría

No reemplazaría:

- medición de superficies
- medición de anchos
- distancias
- espesores

Porque tu parser vectorial probablemente seguirá siendo mucho más preciso.

Si PyMuPDF dice:

muro = 152.34 mm

y FloorplanVLM dice:

≈150 mm

yo confiaría siempre en PyMuPDF.

Donde SÍ confiaría en FloorplanVLM

En la parte semántica.

Ejemplo:

Parser

↓

recinto desconocido

FloorplanVLM

↓

Kitchen

O

Parser

↓

polígono

FloorplanVLM

↓

Eso sí aporta muchísimo valor.

Mi mayor preocupación

Aquí hay un problema importante.

Hoy (agosto de 2026) no encontré evidencia de que FloorplanVLM tenga:

- pesos públicos descargables,
- un repositorio oficial,
- una API pública,
- una demo online.

Encontré el paper y la descripción técnica, pero no una implementación utilizable.  a... +1

Por eso, aunque conceptualmente creo que es una de las arquitecturas más prometedoras para ArchiCheck, **todavía no lo consideraría un componente integrable.**

Lo que yo haría si estuviera liderando ArchiCheck

En vez de intentar incorporar FloorplanVLM inmediatamente, tomaría su arquitectura como referencia y construiría un objetivo interno:

1. Mantener el parser vectorial como fuente de geometría de alta precisión.
2. Añadir un modelo secuencia→JSON (cuando FloorplanVLM publique código o usando un modelo equivalente) como segunda fuente independiente.
3. Definir un **Canonical FloorPlan JSON** propio de ArchiCheck, inspirado en IFC pero simplificado para normativa chilena.
4. Crear un **Consistency Engine** que compare todas las fuentes (parser vectorial, VLM, Claude/GPT y validación del arquitecto) y asigne un nivel de confianza a cada objeto.

Creo que esa arquitectura aprovecharía lo mejor de ambos enfoques: la precisión métrica del parser determinista y la capacidad de generalización semántica de un VLM, sin que tu motor normativo dependa nunca de un proveedor o modelo específico.

